

Title: Mable Technical Lead Engineer - Technical Task

Deadline for Submission: EOD (Sunday 05.07.2026, 11:59 p.m IST), a good week from the date you receive this task. The official start date of the assignment is agreed upon in advance.

Prerequisites

- Clear any doubts with Horst Wenske (horst@mable.ai)

What are you going to do?

For this task, you will build and deploy a small full-stack system and a reusable Go library, benchmark the library, and — as the Technical Lead we are hiring — document the decisions and trade-offs behind it.

Suggested Stack (you can make your tech stack decisions):

- FE
 - React Router 7 (Remix) & React 19
 - Zustand (for FE State Management)
 - Javascript for Browser APIs
 - Tailwind CSS
 - Vite (for bundling)
 - PNPM (for package management)
- BE
 - Go
 - Gin Gonic API Framework
 - An analytics database (e.g. Clickhouse)
- Deployment
 - Cloudflare Workers or Pages (Pages is fine for the Remix frontend)
 - API deployment platform (your choice)) — the Go API may be deployed separately (e.g. Render, Fly.io, Railway); you are not required to run it on Cloudflare Workers
- Observability
 - Grafana

Task List:

1. How do you want to apply AI Coding for the task?
 - a. Frame this as a Technical Lead would for the team: what AI coding would you let an engineer do unsupervised, what must go through review, and how do you keep the quality bar? (The three points below — tooling, guardrails, scope — are the dimensions to cover.)
 - b. Tooling
 - c. Guardrails
 - d. Scope of usage
2. Build a **data pipeline** as a full-stack system plus a reusable library

- Build a demo e-commerce application (can be an Single Page Application use a random free API to populate with dummy products) in Remix
 - Create basic, but functional UI/UX for
 - Sign Up and Login Page
 - Product Library
 - Cart Operations
 - Checkout
 - Style using Tailwind CSS (use React Icons and other free graphics for ease)
 - Simulate checkout flow, and return user to product library when successful
 - Uses the Go API (mentioned below) to serve a simple sign-up/login flow using **JWT-based authentication, with the token stored in an HttpOnly cookie**
 - Apply frontend best practices throughout: accessibility (semantic HTML, labels, keyboard navigation), form validation with clear error messaging, and explicit loading / empty / error states for every data-driven view.
 - Keep tracking calls non-blocking (they must never block or break the UI), and do not store sensitive auth state (e.g. JWTs) in client-accessible storage such as localStorage.
 - Include at least one moderately complex UI workflow — e.g. a multi-step checkout, offline/retry behaviour for failed requests, or session restoration after a page refresh.
 - Add frontend observability: React error boundaries, client-side error tracking, structured logging, and a short note on your monitoring strategy for the FE.
 - Describe (and where practical, implement) a frontend testing strategy across unit, integration, and end-to-end levels, stating what should and should not be tested. A basic end-to-end test with a tool such as Playwright is a plus.
 - In DESIGN.md, explain how you would scale the frontend codebase for multiple engineers and future features (structure, state management, conventions, code ownership).
 -
- On this demo e-commerce application, implement a tracking script.
 - Use the Standard Web APIs (<https://developer.mozilla.org/en-US/docs/Web/API>) to build the tracking logic and data layer.
 - Ensure that your script can track user events like:
 - Clicks
 - Page Views
 - Add To Cart
 - Checkout
 - Payment Info Added
 - Purchase
 - Lead (Email Form Submission)
 - **[OPTIONAL]** - Can you think of any other events that are valuable for user tracking? Implement any that you can think of
 - Track user data parameters like
 - User Agent
 - IP Address
 - CartData and other Details from Checkout Form
 - Userdata

- Location
 - Timezone
 - Details from logged-in user session
- Tracked events need to be sent to an API Service written in Go
- Build a Go API with the following functionality in Gin:
 - Can receive events from the **tracking script**
 - Process received events through the pipeline library (below) before persistence: every event the API ingests must flow through a pipeline.Pipeline[Event] — at minimum validation/normalisation and enrichment stages — and only the pipeline output is written to the analytics DB. The library is a working part of the ingest path, not a standalone demo.
 - Serves basic authentication (signup and login) via **JWT Tokens** issued in an HttpOnly cookie (state your CORS and cross-site cookie strategy, since the SPA and API are on different origins)
 - Serves a liveness/readiness check at a standard endpoint (**/health**) — keep it lightweight (200 + minimal JSON); if you expose Prometheus-style metrics, serve them separately at **/metrics**
 - Dumps tracked event metadata to **the analytics DB**.
 - Calculate statistics such as
 - Average event capture time
 - Average event parameters
 - Events tracked over time
 - Event Statistics
 - Event counts over time for each event type
 - **[OPTIONAL]** - Any other stats you can think of
- **[OPTIONAL]** Create a visualization in Grafana to visualize the different analytics data crunched in the previous step
 - Make the dashboard reproducible — a runnable docker compose up plus screenshots is sufficient; a publicly accessible link (e.g. Grafana Cloud) is a nice-to-have, not required
 - Create a standard view featuring the **3 most important statistics you choose** in relevant graphs
- Deploy your applications, and record a demo video walkthrough of you demonstrating the functionality of the **data pipeline** on deployed versions of the individual applications
- **[OPTIONAL]** Profile your apps, BE and FE, and send in benchmarks as well - use industry standard best practices for profiling your FE and BE apps.
- Build a generic, concurrent data-pipeline library in Go (systems-design deliverable)
 - As a Technical Lead, we also want to see how you design a reusable library, not just wire services together. Build a small but production-minded streaming pipeline library in Go that the tracking API above uses to process events.
 - Integrate this library into the Go API's event-ingest path described above (not only the benchmark harness): the API must consume the pipeline to process events on the way to the analytics DB, and the per-stage metadata below comes from that same in-API pipeline run. Treat the standalone benchmark and the in-API integration as two callers of one library.
 - A pipeline is generic over a single element type — Pipeline[T] — and is composed of stages. Support these stage types:

- `Map[T]` — transforms a `T` into a `T` (`func(T) T`).
- `Filter[T]` — drops events for which a predicate is false (`func(T) bool`).
- `Generate[T]` — a $1 \rightarrow N$ stage: given one `T`, emits the original plus zero-or-more newly produced `T` downstream.
- `If[T]` — routes each `T` into one of two sub-pipelines (both `Pipeline[T]`); their outputs merge back into the downstream `T` stream.
- `Reduce[T,R]` — terminates a pipeline by folding a stream of `T` into a single (or keyed) `R` sink. Reduce is a sink: it is not chainable into further `T` stages. State whether your reduce is per-batch or global, and why.
- `Collect[T]` — drains the stream into a caller-provided, bounded sink (document its back-pressure behaviour).
- Decide the typing model and document the trade-off: a homogeneous element type (`Pipeline[T]`) vs. type-erasure (any) at stage boundaries are both acceptable — we care that you make the call and justify it, not that you pick a particular one.
- The library must support dynamic batching and fan-out across worker goroutines, with the most important hyperparameters configurable (e.g. batch size, worker count, channel buffer depth).
 - **[OPTIONAL]** - Make one or more hyperparameters self-tuning from runtime signals (e.g. adjust worker count from observed queue depth).
- Provide a protocol/interface for future developers to add new stage types without modifying the core — adding a stage should be trivial, and demonstrate it with at least one example stage.
- Tests and benchmarks:
 - Unit-test the library and keep it clean under the race detector (go test -race). We value meaningful tests over a coverage number; aim for solid coverage of the pipeline package (excluding the test harness and main glue).
 - Benchmark the pipeline across increasing event volumes — 10, 1k, 100k, 1M events — for two payloads: a fixed benchmark struct (below) and the sample Mable event, committed to the repo as `sample_event.json` (a real, valid JSON file — not pasted into this document).
 - Use one fixed struct for the synthetic payload so numbers are comparable: a `TestStruct` with 10 fields of mixed, pinned types. Use the definition provided in the starter if present; otherwise define one and state the field types. We judge methodology, not absolute throughput.
 - Report each benchmark with its method and the machine it ran on (CPU, cores, RAM). Keep a default cap of 1M events so the run stays laptop-safe.
 - **[OPTIONAL]** - Push to 10M events with the sink streamed to disk (do not retain all events in memory), and/or experiment with Go GC tuning and binary build strategies — sequence these after the mandatory benchmarks.
- Have the library emit metadata per stage — not just one timing for the whole pipeline — (e.g. per-stage latency, batch size, throughput, and error/drop

counts) and ingest it into the analytics DB alongside the tracking events. Emitting per-stage errors and dropped events, not only latency, is required.

- **[OPTIONAL]** - Surface this pipeline metadata on the same Grafana dashboard as the tracking analytics (the dashboard itself is stretch — see the time note below).
- Write a short DESIGN.md (1–2 pages) — we weight this heavily for a Technical Lead.
 - Cover the architecture, the 3–4 decisions that actually mattered, and the alternatives you considered and rejected.
 - Call out the trade-offs you made (e.g. the auth model against the cross-origin constraint, and the pipeline's typing model) and defend them.
 - Note the failure modes you are aware of and how you would scale the system.
- Include a short written code review in REVIEW.md (a Technical Lead reviews more than they author).
 - Pick one non-trivial file from your own submission (a Go handler or a React component) and review it as if a teammate wrote it: list what you would change and why, severity-ranked, and what you would accept as-is.
- In DESIGN.md, add one paragraph: how would you split this work across a team of three and sequence the first two weeks?

TIME EXPECTATION - This is a one-week task and the brief is deliberately broad. The must-haves are the e-commerce app, the tracking script, the Go event API with ingest to the analytics DB, and the pipeline library with its tests, its 1M-event benchmarks, and its per-stage metadata ingested to the analytics DB. Grafana dashboards, a publicly accessible dashboard link, the 10M / GC experiments, and profiling are stretch (OPTIONAL). Prioritise correctness, clear design decisions, and the library over breadth of OPTIONAL items.

IMPORTANT - Any tasks marked **[OPTIONAL]** are not mandatory to complete but will add extra credit to your case if you manage to complete them.

However, you will **NOT** be penalized for not completing them.

How will you submit the tasks?

- Once the tasks are completed, please push to a public repository on your own GitHub with the following naming convention -
'<insert_name_here>_mable_technical_lead_task_<date_of_submission>'
- Upload all code to the following subfolders
 - a. **ecommerce** for the e-commerce application
 - b. **script** for the tracking script
 - c. **api** for the Go API
 - d. **pipeline** for the Go pipeline library
 - e. **links** for links to the **walkthrough video** and **grafana dashboard**
- Share the link to the repository with us
 - a. horst@mable.ai, sayak@mable.ai, malay@mable.ai

Ground rules (please read before you start):

- AI coding tools are allowed and encouraged — please disclose how you used them (see the AI-coding question above).
- Use synthetic data only — do not collect data from real users. In production the tracked parameters (IP, location, checkout details, session) require consent and data-retention limits; add a simple consent gate and/or note where consent would be required, and avoid persisting real PII.
- The repository must be public, and any change after the submission deadline has to be communicated in advance.

Evaluation Criteria:

- Your technical submission will be evaluated on the following criteria:
- The rubric is weighted; percentages show how much each axis counts toward the score.
 - Design judgment & communication (25%) — DESIGN.md names the decisions that mattered, the alternatives rejected, and the failure modes; trade-offs (auth, pipeline typing) are argued, not just made.
 - Library & systems design (20%) — clean stage interface; the Pipeline[T] / Reduce[T,R] typing decision is justified; adding a stage is trivial and demonstrated; concurrency is race-free with bounded memory and back-pressure at scale. The pipeline is wired into the API's ingest path (not a standalone demo), and the per-stage metadata is produced by that in-API pipeline.
 - Code quality & tests (20%) — -race clean; meaningful tests over coverage %; readable Gin handlers; one coherent auth model.
 - Product/full-stack execution (15%) — the core user journey works end-to-end; the browser-vs-server tracking split is correct; sensible event and data model.
 - Leadership & risk instinct (15%) — the code-review artifact is sharp and severity-ranked; the candidate proactively flags Personal Identity Information/consent; the team-split answer is realistic.
 - Creativity / stretch (5%) — OPTIONAL items, extra event types, profiling, self-tuning, or an unprompted product/ops insight.